

COMPLETE PROJECT GUIDE - Everything Explained

PROJECT OVERVIEW

DrugGuard - Drug Interaction Checker is a hybrid AI-powered web application that combines:

- Rule-based clinical logic (primary) for diabetic patients
- Machine Learning predictions (supplementary) from TWOSIDES database
- OCR image recognition for medication labels
- Real-time drug risk assessment for diabetic patients with kidney disease

ROOT DIRECTORY STRUCTURE

```
C:\Drug\  
├─ backend/          # Python FastAPI backend server  
├─ frontend/         # React + Vite frontend application  
├─ README.md         # Project overview and quick start  
├─ PROJECT_ABSTRACT.md      # Project abstract and title  
├─ PROJECT_STRUCTURE_EXPLAINED.md # Previous structure explanation  
├─ COMPLETE_PROJECT_GUIDE.md # This file!  
├─ run_backend.bat    # Windows script to start backend  
├─ run_frontend.bat   # Windows script to start frontend  
└─ start_app.bat      # Windows script to start both
```

BACKEND DIRECTORY (backend/)

Core Application (backend/app/)

main.py (916 lines) - THE HEART OF THE BACKEND

- **Purpose:** FastAPI application entry point
- **What it does:**
 - Creates the FastAPI app instance
 - Sets up CORS (Cross-Origin Resource Sharing) for frontend communication
 - Defines all API endpoints:
 - /drugs/search - Search drugs by name

- `/drugs` - List all drugs with pagination
- `/drugs/{id}` - Get drug by ID
- `/interactions/check` - Check interaction between two drugs
- `/ocr/extract` - Extract drug names from images
- `/health` - Health check endpoint
- `/stats` - Database statistics
- Includes middleware for logging drug searches
- Initializes database on startup
- Mounts diabetic patient router (`/diabetic/*`)

config.py - Configuration Settings

- **Purpose:** Centralized configuration management
- **What it does:**
 - Loads environment variables
 - Database connection settings
 - API keys and secrets
 - Feature flags

database.py - Database Setup

- **Purpose:** SQLAlchemy database connection and session management
- **What it does:**
 - Creates async database engine (SQLite)
 - Provides `async_session` for database operations
 - Initializes database tables (`init_db()`)
 - Database connection pooling

models.py - Database Models (SQLAlchemy ORM)

- **Purpose:** Defines database table structures
- **Tables:**
 - `Drug` - Stores drug information (name, class, mechanism)
 - `DrugInteraction` - Stores drug-drug interactions
 - `Category` - Drug categories
 - `ComparisonLog` - Logs ML vs rule-based comparisons
 - `DiabeticPatient` - Diabetic patient profiles
 - `DiabeticMedication` - Patient medications

schemas.py - Pydantic Schemas

- **Purpose:** Request/response validation and serialization

- **What it does:**
 - Defines data structures for API requests/responses
 - Validates incoming data
 - Converts between Python objects and JSON

Diabetic Patient Module (`backend/app/diabetic/`)

`rules.py` - CLINICAL RULE ENGINE ⚠️ CRITICAL

- **Purpose:** Rule-based drug risk assessment for diabetic patients
- **What it does:**
 - Contains **70+ specific drug rules** with eGFR thresholds
 - **Drug class patterns** (e.g., `-pril` , `-sartan` , `-olol`) for entire classes
 - **eGFR-based contraindications:**
 - eGFR < 15: Fatal risk drugs
 - eGFR < 30: Severe CKD warnings
 - eGFR < 45: Moderate CKD adjustments
 - **Key functions:**
 - `assess_drug_risk()` - Main risk assessment function
 - Checks specific drugs → drug classes → general CKD warnings
 - **Priority:** Rules are PRIMARY, ML is supplementary

`service.py` - Diabetic Patient Service

- **Purpose:** Business logic for diabetic patient management
- **What it does:**
 - Creates/manages diabetic patient profiles
 - Adds medications to patients
 - **`check_drug_risk()`** - Main function that:
 - a. Gets rule-based assessment (PRIMARY)
 - b. Gets ML prediction (supplementary)
 - c. Returns combined result (rules override ML)
 - Generates patient reports
 - Checks all medications for a patient

`router.py` - API Routes for Diabetic Features

- **Purpose:** FastAPI routes for diabetic patient endpoints
- **Endpoints:**
 - `POST /diabetic/patients` - Create patient

- GET /diabetic/patients - List all patients
- GET /diabetic/patients/{id} - Get patient details
- POST /diabetic/patients/{id}/medications - Add medication
- POST /diabetic/risk-check - Check drug risk for patient
- GET /diabetic/report/{id} - Generate patient report

schemas.py - Diabetic Patient Data Schemas

- **Purpose:** Pydantic schemas for diabetic patient data
- **Schemas:**
 - DiabeticPatientCreate - Create patient request
 - DiabeticPatientResponse - Patient response
 - MedicationCreate - Add medication request
 - DrugRiskCheckRequest - Drug risk check request
 - DrugRiskCheckResponse - Risk assessment result

models.py - Diabetic Database Models

- **Purpose:** SQLAlchemy models for diabetic patients
- **Tables:**
 - DiabeticPatient - Patient profiles with labs, complications
 - DiabeticMedication - Patient medication list

validation.py - Data Validation

- **Purpose:** Validates diabetic patient data
- **What it does:**
 - Validates eGFR ranges
 - Validates lab values
 - Validates medication data

m1_predictor.py - ML Predictor for Diabetic Module

- **Purpose:** Wrapper for ML predictions in diabetic context
- **What it does:**
 - Calls main ML predictor
 - Formats results for diabetic patient context

data/diabetes_medications.json - Diabetes Medication List

- **Purpose:** List of diabetes-specific medications
- **What it contains:**
 - Common diabetes drugs (Metformin, Insulin, etc.)

- Used for identifying diabetes medications

Machine Learning Module (backend/app/ml/)

`predictor.py` (368 lines) - ML PREDICTION ENGINE

- **Purpose:** Loads trained ML models and makes predictions
- **What it does:**
 - Loads 3 models: Random Forest, XGBoost, LightGBM
 - Creates ensemble predictions (average of 3 models)
 - Uses **optimal threshold** (from `optimal_threshold.json`) instead of 0.5
 - Feature engineering (hash encoding of drug names/classes)
 - Returns probability and binary prediction

`feature_engineering.py` - Feature Extraction

- **Purpose:** Converts drug names to ML features
- **What it does:**
 - Hash encoding of drug names
 - Drug class encoding
 - Mechanism of action encoding
 - Creates numerical feature vectors for ML models

`models.py` - ML Model Definitions

- **Purpose:** Model wrapper classes
- **What it does:**
 - `DDIModel` - Wrapper for saved models
 - Model metadata storage

`trainer.py` - Model Training Logic

- **Purpose:** Training pipeline for ML models
- **What it does:**
 - Cross-validation
 - Hyperparameter tuning
 - Model evaluation

`bayesian_optimizer.py` - Hyperparameter Optimization

- **Purpose:** Uses Optuna for hyperparameter tuning
- **What it does:**

- Bayesian optimization
- Finds best hyperparameters
- Reduces overfitting

Services (backend/app/services/)

interaction_service.py - Core Interaction Logic

- **Purpose:** Main service for drug interaction checking
- **What it does:**
 - Searches database for interactions
 - Determines severity levels
 - Finds safe alternatives

ocr_service.py (415 lines) - IMAGE RECOGNITION

- **Purpose:** Extracts drug names from images
- **What it does:**
 - **8 preprocessing variants:**
 - a. Original grayscale
 - b. CLAHE + adaptive threshold
 - c. High contrast (OTSU)
 - d. Inverted (dark backgrounds)
 - e. 2x rescaled (small text)
 - f. Sharpened
 - g. Bilateral filter (noise reduction)
 - h. Morphological operations
 - **7 Tesseract configurations** per variant (56 total attempts!)
 - **Drug name extraction:**
 - Regex patterns for drug suffixes (-pril , -sartan , etc.)
 - Brand name detection (ALL CAPS)
 - Dosage extraction
 - Filters non-drug words
 - **Fuzzy matching** against database (similarity threshold: 0.5)
 - Returns detected drug names with confidence scores

comparison_logger.py - ML vs Rules Logging

- **Purpose:** Logs when ML and rules disagree
- **What it does:**

- Records ML predictions
- Records rule-based decisions
- Stores disagreements for analysis
- Helps improve ML models

`data_fetcher.py` - External Data Collection

- **Purpose:** Fetches drug data from external APIs
- **What it does:**
 - OpenFDA API
 - RxNorm (NIH)
 - DrugBank
 - Saves to database

`cache.py` - Caching Service

- **Purpose:** Caches frequently accessed data
- **What it does:**
 - In-memory caching
 - Reduces database queries
 - Improves performance

`rate_limiter.py` - API Rate Limiting

- **Purpose:** Prevents API abuse
- **What it does:**
 - Limits requests per IP
 - Protects against DDoS

`auth.py` - Authentication

- **Purpose:** API key authentication
- **What it does:**
 - Validates API keys
 - Protects admin endpoints

`tasks.py` - Background Jobs

- **Purpose:** Async task processing
- **What it does:**
 - Enqueues ML training jobs
 - Data refresh tasks
 - Uses Redis Queue (RQ)

Scripts (backend/scripts/)

Data Loading Scripts

`load_twosides.py` - Load TWOSIDES Database

- **Purpose:** Loads TWOSIDES drug interaction database
- **What it does:**
 - Reads `TWOSIDES.csv.gz` (42M+ interactions)
 - Inserts into `twosides_interactions` table
 - Progress tracking

`load_offsides.py` - Load OFFSIDES Database

- **Purpose:** Loads OFFSIDES database
- **What it does:**
 - Similar to TWOSIDES
 - Alternative data source

`map_twosides_severity.py` - Map Severity Levels

- **Purpose:** Maps severity to TWOSIDES interactions
- **What it does:**
 - Uses cursor-based pagination (fast!)
 - Batch updates (10k rows at a time)
 - Maps: minor, moderate, major, contraindicated
 - **Optimized:** No OFFSET, uses `WHERE id > last_id`

`map_offsides_severity.py` - Map OFFSIDES Severity

- **Purpose:** Similar to TWOSIDES severity mapping

Training Set Building Scripts

`build_training_set.py` - Build ML Training Data

- **Purpose:** Creates train/val/test splits from TWOSIDES
- **What it does:**
 - **Streaming approach** (no memory issues!)
 - Processes 50k rows at a time
 - Generates positive samples (known interactions)
 - Generates negative samples (no interaction)
 - Writes directly to CSV files

- **Arguments:**
 - `--max-positives` - Limit positive samples
 - `--negatives` - Number of negative samples
 - `--batch-size` - Processing batch size

`build_diabetic_pseudolabels.py` - Build Diabetic Training Data

- **Purpose:** Creates pseudo-labeled data from MIMIC-IV
- **What it does:**
 - Loads MIMIC-IV demo data
 - Builds patient contexts (age, labs, complications)
 - Applies rule engine to generate labels
 - Creates train/val/test splits
 - Saves to `data/diabetic/training/`

Model Training Scripts

`train_twosides_ml.py` (657 lines) - MAIN ML TRAINING

- **Purpose:** Trains ML models on TWOSIDES data
- **What it does:**
 - Loads training data
 - **Feature engineering** (hash encoding)
 - Trains 3 models:
 - Random Forest (`class_weight='balanced'`)
 - XGBoost (`scale_pos_weight`)
 - LightGBM (`scale_pos_weight`)
 - **Resampling options** (`--resampling`):
 - `smote` - SMOTE oversampling
 - `adasyn` - ADASYN oversampling
 - `borderline` - BorderlineSMOTE
 - `svm` - SVMSMOTE
 - `tomek` - SMOTETomek
 - `enn` - SMOTEENN
 - **Arguments:**
 - `--max-samples` - Limit training samples
 - `--resampling` - Resampling technique
 - `--sampling-ratio` - Target class balance
 - `--fast` - Quick training mode
 - Saves models to `models/`
 - Evaluates on validation set

`train_diabetic_ml.py` - Train Diabetic-Specific Models

- **Purpose:** Trains models on diabetic pseudo-labels
- **What it does:**
 - Similar to `train_twosides_ml.py`
 - Uses diabetic training data
 - Saves to `models/diabetic_risk_model.pkl`

`retrain_with_imbalanced_learn.py` - Convenience Script

- **Purpose:** Quick retraining with SMOTE
- **What it does:**
 - Calls `train_twosides_ml.py` with SMOTE enabled
 - Improves NPV (Negative Predictive Value)

Model Evaluation Scripts

`evaluate_models.py` (602 lines) - MODEL EVALUATION

- **Purpose:** Evaluates trained models
- **What it does:**
 - Loads test set
 - Evaluates all 3 models + ensemble
 - **Metrics:**
 - Accuracy, Precision, Recall, F1-Score
 - **Specificity, Sensitivity, PPV, NPV** (critical for medical!)
 - AUC-ROC
 - Confusion matrix
 - `--find-optimal` : Finds optimal threshold
 - Saves results to `models/evaluation_results.json`

`find_optimal_threshold.py` (228 lines) - THRESHOLD OPTIMIZATION

- **Purpose:** Finds best classification threshold
- **What it does:**
 - Loads balanced test sample (10k positive, 10k negative)
 - Computes ensemble probabilities
 - **Two methods:**
 - **G-Mean:** $\sqrt{\text{Sensitivity} * \text{Specificity}}$
 - **Youden's J:** $\text{Sensitivity} + \text{Specificity} - 1$
 - Compares thresholds: 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8
 - Saves optimal threshold to `models/optimal_threshold.json`

- **Critical:** Improves NPV from 11% to 99%!

Utility Scripts

`seed_demo_patients.py` - Seed Demo Data

- **Purpose:** Creates demo diabetic patients
- **What it does:**
 - Creates 5 demo patients:
 - DEMO001 - John Smith (eGFR: 45)
 - DEMO002 - Sarah Johnson (eGFR: 78)
 - DEMO003 - Michael Chen (eGFR: 28, severe CKD)
 - DEMO004 - Emily Davis (eGFR: 92)
 - DEMO005 - Robert Williams (eGFR: 35)
 - Adds medications to each patient
 - Handles existing patients gracefully

`fetch_real_data.py` - Fetch External Data

- **Purpose:** Fetches drug data from APIs
- **What it does:**
 - OpenFDA
 - RxNorm
 - DrugBank

`rq_worker.py` - Background Worker

- **Purpose:** Processes background jobs
- **What it does:**
 - Runs Redis Queue worker
 - Processes ML training jobs
 - Data refresh tasks

Data Directory (`backend/data/`)

`interactions.db` - Main SQLite Database

- **Purpose:** Stores all drug and interaction data
- **Tables:**
 - `drugs` - Drug information
 - `drug_interactions` - Interactions
 - `twosides_interactions` - TWOSIDES data

- `diabetic_patients` - Patient profiles
- `diabetic_medications` - Patient meds
- `comparison_logs` - ML audit logs

`drug_interactions.db` - Alternative Database

- **Purpose:** Backup or alternative database

`training/` - ML Training Data

- **Files:**
 - `train.csv` - Training set (80%)
 - `val.csv` - Validation set (10%)
 - `test.csv` - Test set (10%)
 - `metadata.json` - Dataset metadata

`diabetic/training/` - Diabetic Training Data

- **Files:**
 - `train.csv` , `val.csv` , `test.csv`
 - `full_dataset.csv` - Complete dataset
 - `metadata.json` - Metadata

`twosides/TWOSIDES.csv.gz` - TWOSIDES Source Data

- **Purpose:** Original TWOSIDES database (42M+ rows)
- **Size:** ~2GB compressed

`offsides/OFFSIDES.csv` - OFFSIDES Source Data

- **Purpose:** Alternative interaction database

`mimiciv/demo/` - MIMIC-IV Demo Data

- **Purpose:** Hospital patient data for pseudo-labeling
- **What it contains:**
 - Patient demographics
 - Lab results
 - Prescriptions
 - Diagnoses (ICD codes)

`real_drug_data.json` - Cached Drug Data

- **Purpose:** Cached drug information from APIs

Models Directory (backend/models/)

Trained ML Models (.pk1 files)

- `random_forest_model.pk1` - Random Forest model
- `xgboost_model.pk1` - XGBoost model
- `lightgbm_model.pk1` - LightGBM model
- `diabetic_risk_model.pk1` - Diabetic-specific model
- `feature_extractor.pk1` - Feature engineering pipeline

Model Metadata (.json files)

- `optimal_threshold.json` - Optimal classification threshold

```
{
  "threshold": 0.2345,
  "method": "gmean",
  "auc_roc": 0.957,
  "test_samples": 20000
}
```

- `evaluation_results.json` - Model evaluation metrics
- `training_results.json` - Training history
- `*_comparison.json` - Model comparison results

Other Backend Files

`requirements.txt` - Python Dependencies

- **Core:** FastAPI, Uvicorn, SQLAlchemy
- **ML:** scikit-learn, XGBoost, LightGBM, Optuna, imbalanced-learn
- **Data:** pandas, numpy
- **OCR:** opencv-python, pytesseract, Pillow
- **Utils:** pydantic, httpx, python-dotenv

`MODEL_ACCURACY_REPORT.md` - Accuracy Report

- **Purpose:** Documents model performance
- **What it contains:**
 - Accuracy metrics
 - Class imbalance analysis
 - Threshold optimization results

logs/comparison_history.json - Comparison Logs

- **Purpose:** Logs ML vs rules disagreements
- **What it contains:**
 - Drug pairs checked
 - ML predictions
 - Rule-based decisions
 - Disagreements

FRONTEND DIRECTORY (frontend/)

Source Code (frontend/src/)

main.jsx - Application Entry Point

- **Purpose:** React application bootstrap
- **What it does:**
 - Renders <App /> component
 - Sets up React Router
 - Applies global CSS

App.jsx - Main Application Component

- **Purpose:** Root component with routing
- **What it does:**
 - Defines routes:
 - / - Home page (InteractionChecker)
 - /diabetes - Diabetes Manager
 - /ml-dashboard - ML Dashboard
 - Navigation between pages

index.css - Global Styles

- **Purpose:** Tailwind CSS imports and global styles
- **What it does:**
 - Imports Tailwind directives
 - Custom CSS variables
 - Dark theme colors

Components (frontend/src/components/)

Navbar.jsx - Navigation Bar

- **Purpose:** Top navigation bar
- **What it does:**
 - Logo and branding
 - Navigation links (How it Works, Features)
 - Buttons: "Diabetes DDI", "ML Dashboard", "System Online"

Hero.jsx - Hero Section

- **Purpose:** Landing page hero section
- **What it does:**
 - Welcome message
 - Call-to-action buttons
 - Visual design

InteractionChecker.jsx - Main Drug Checker

- **Purpose:** Primary drug interaction checker
- **What it does:**
 - Two drug input fields with autocomplete
 - Search functionality
 - Displays interaction results
 - Shows severity badges
 - Safe alternatives display

DiabetesManager.jsx - DIABETIC PATIENT MANAGER ⚠️ CRITICAL

- **Purpose:** Complete diabetic patient management interface
- **What it does:**
 - **Patient Management:**
 - Create new patients
 - Select existing patients
 - View patient details (eGFR, labs, complications)
 - **Medication Management:**
 - Add medications to patient
 - View patient's medication list
 - "Check Patient Meds" button (checks all meds)
 - **Drug Risk Checking:**
 - Enter drug name to check
 - "Check Drug Risk" button
 - Displays risk level, severity, explanation

- Shows ML prediction (supplementary)
- **Browse All Drugs:**
 - Modal with all 8,263 drugs
 - Pagination (50 per page)
 - Search functionality
 - Click to select drug
- **Patient Reports:**
 - Generate comprehensive report
 - Download as JSON
- **Visual Features:**
 - Risk badges (Safe, Caution, High Risk, Contraindicated, Fatal)
 - ML badges (shows ML probability)
 - Patient cards
 - Medication list

ResultsDisplay.jsx - Interaction Results

- **Purpose:** Displays interaction check results
- **What it does:**
 - Severity badges
 - Interaction description
 - Recommendations
 - Visual indicators

AlternativesDisplay.jsx - Safe Alternatives

- **Purpose:** Shows safe alternative drugs
- **What it does:**
 - Lists alternative drugs
 - No interaction guarantee
 - Click to select alternative

CameraCapture.jsx - Camera/OCR Component

- **Purpose:** Image capture and OCR
- **What it does:**
 - Webcam access
 - Image capture
 - Upload image
 - Calls OCR API
 - Displays detected drugs

MLPrediction.jsx - ML Prediction Display

- **Purpose:** Shows ML model predictions
- **What it does:**
 - Displays ML probability
 - Model confidence
 - Ensemble prediction

ModelDashboard.jsx - ML Model Dashboard

- **Purpose:** ML model performance dashboard
- **What it does:**
 - Model metrics (Accuracy, Precision, Recall, F1)
 - Confusion matrix visualization
 - Threshold optimization results
 - Model comparison charts

FloatingElements.jsx - Animated Background

- **Purpose:** Decorative floating elements
- **What it does:**
 - Animated background shapes
 - Visual appeal

Footer.jsx - Footer Component

- **Purpose:** Page footer
- **What it does:**
 - Copyright information
 - Links
 - Disclaimer

Services (frontend/src/services/)

api.js - API Client

- **Purpose:** Centralized API communication
- **What it does:**
 - `searchDrugs()` - Search drugs
 - `checkInteraction()` - Check drug interaction
 - `getAlternatives()` - Get safe alternatives
 - `extractFromImage()` - OCR extraction

- `getMLPrediction()` - ML prediction
- `getMLModelInfo()` - Model information
- Error handling
- Base URL: `http://localhost:8001` (or from env)

Utils (`frontend/src/Utils/`)

`debounce.js` - Debounce Utility

- **Purpose:** Delays function execution
- **What it does:**
 - Prevents excessive API calls
 - Used in search autocomplete

Configuration Files

`package.json` - Node.js Dependencies

- **Dependencies:**
 - `react`, `react-dom` - React framework
 - `react-router-dom` - Routing
 - `framer-motion` - Animations
 - `react-webcam` - Camera access
 - `react-hot-toast` - Notifications
 - `lucide-react` - Icons
- **Dev Dependencies:**
 - `vite` - Build tool
 - `tailwindcss` - CSS framework
 - `@vitejs/plugin-react` - React plugin

`vite.config.js` - Vite Configuration

- **Purpose:** Vite build configuration
- **What it does:**
 - React plugin
 - Port configuration
 - Proxy settings

`tailwind.config.js` - Tailwind CSS Configuration

- **Purpose:** Tailwind CSS customization
- **What it does:**
 - Theme colors
 - Custom utilities
 - Dark mode

`postcss.config.js` - PostCSS Configuration

- **Purpose:** CSS processing
- **What it does:**
 - Autoprefixer
 - Tailwind CSS

`index.html` - HTML Entry Point

- **Purpose:** HTML template
- **What it does:**
 - Root div for React
 - Meta tags
 - Title

`public/pill.svg` - Logo

- **Purpose:** Application logo
- **What it is:** SVG pill icon



HOW IT ALL WORKS TOGETHER

1. User Flow: Check Drug Risk for Diabetic Patient

1. Frontend (`DiabetesManager.jsx`):

- User selects patient (e.g., DEMO003 with eGFR=28)
- User enters drug name (e.g., "Verapamil")
- Clicks "Check Drug Risk"

2. API Request (`api.js`):

- `POST /diabetic/risk-check`
- Body: `{ patient_id: "DEMO003", drug_name: "Verapamil" }`

3. Backend Router (`diabetic/router.py`):

- Receives request
- Calls `service.check_drug_risk()`

4. Service Layer (`diabetic/service.py`):

- Gets patient from database
- Gets patient's current medications
- **Calls Rule Engine (`rules.py`):**
 - Checks if Verapamil is in `EGFR_CONTRAINDICATIONS`
 - Finds: `"Verapamil": {"egfr_threshold": 30, "severity": "contraindicated"}`
 - Patient `eGFR=28 < 30` → **CONTRAINDICATED**
- **Calls ML Predictor (`ml/predictor.py`):**
 - Loads 3 models
 - Gets ensemble probability: 0.15 (low risk)
 - Uses optimal threshold: 0.2345
 - Prediction: Safe (`0.15 < 0.2345`)
- **Rules Override ML:**
 - Final decision: **CONTRAINDICATED** (from rules)
 - ML result stored but not used

5. Response:

```
{
  "risk_level": "contraindicated",
  "severity": "contraindicated",
  "risk_score": 0.95,
  "explanation": "Verapamil is contraindicated in patients with eGFR < 30...",
  "ml_probability": 0.15,
  "ml_risk_level": "safe",
  "rule_override": true
}
```

6. Frontend Display:

- Shows red "× Contraindicated" badge
- Displays explanation
- Shows ML prediction (gray badge, supplementary)

2. ML Training Pipeline

1. Data Loading (`load_twosides.py`):

- Loads 42M TWOSIDES interactions

- Maps severity (`map_twosides_severity.py`)

2. Training Set (`build_training_set.py`):

- Creates positive samples (known interactions)
- Creates negative samples (no interaction)
- Splits: 80% train, 10% val, 10% test
- Saves to `data/training/`

3. Model Training (`train_twosides_ml.py`):

- Loads training data
- Feature engineering (hash encoding)
- Trains Random Forest, XGBoost, LightGBM
- Uses class weights (`class_weight='balanced'`)
- Optionally uses SMOTE (`--resampling smote`)
- Saves models to `models/`

4. Evaluation (`evaluate_models.py`):

- Tests on test set
- Computes metrics (Accuracy, Precision, Recall, F1, NPV)
- Finds optimal threshold (`find_optimal_threshold.py`)
- Saves threshold to `models/optimal_threshold.json`

5. Production Use (`ml/predictor.py`):

- Loads trained models
- Uses optimal threshold
- Makes predictions

3. OCR Pipeline

1. User Uploads Image (`CameraCapture.jsx`):

- User captures/uploads medication label image

2. API Request (`main.py`):

- `POST /ocr/extract`
- Body: `{ image_base64: "..."` }

3. OCR Service (`services/ocr_service.py`):

- **Preprocessing** (8 variants):
 - Grayscale, CLAHE, OTSU, Inverted, Rescaled, Sharpened, Bilateral, Morphological
- **Tesseract OCR** (7 configs per variant = 56 attempts):
 - Tries different PSM modes
 - Extracts text
- **Drug Name Extraction:**
 - Regex patterns (-pril , -sartan , etc.)
 - Brand name detection
 - Dosage extraction
- **Fuzzy Matching:**
 - Matches against database (8,263 drugs)
 - Similarity threshold: 0.5
 - Returns matched drugs

4. Response:

```
{  
  "detected_drugs": [ "Metformin", "Lisinopril" ],  
  "confidence": [ 0.95, 0.87 ]  
}
```

5. Frontend:

- Displays detected drugs
- User can select drug



DATABASE SCHEMA

Main Tables

drugs

- id (PK)
- name - Drug name
- generic_name - Generic name
- drug_class - Drug class
- mechanism - Mechanism of action

drug_interactions

- id (PK)

- drug1_id (FK)
- drug2_id (FK)
- severity - minor/moderate/major/contraindicated
- description - Interaction description

twosides_interactions

- id (PK)
- drug1_name
- drug2_name
- severity - Mapped severity
- frequency - Interaction frequency

diabetic_patients

- id (PK)
- patient_id (Unique)
- name , age , gender
- diabetes_type - type_1/type_2/prediabetes
- egfr - Estimated GFR
- creatinine , potassium - Lab values
- has_nephropathy , has_retinopathy , etc. - Complications

diabetic_medications

- id (PK)
- patient_id (FK)
- drug_name
- dosage , frequency
- is_active - Currently taking

comparison_logs

- id (PK)
- drug1_name , drug2_name
- rule_decision - Rule-based decision
- ml_probability - ML probability
- ml_decision - ML decision
- rule_override_reason - Why rules overrode ML



STARTING THE APPLICATION

Option 1: Batch Scripts (Windows)

```
# Start backend only
run_backend.bat

# Start frontend only
run_frontend.bat

# Start both
start_app.bat
```

Option 2: Manual

```
# Backend
cd backend
venv\Scripts\activate
uvicorn app.main:app --reload --port 8001

# Frontend (new terminal)
cd frontend
npm run dev
```

Option 3: Python Scripts

```
# Backend
cd backend
python -m uvicorn app.main:app --reload --port 8001
```



KEY METRICS & PERFORMANCE

ML Model Performance (After Optimization)

- Accuracy: 85-96%
- NPV (Negative Predictive Value): 99.41% ⚠️ CRITICAL
- AUC-ROC: 0.957
- Optimal Threshold: 0.2345 (not 0.5!)

Database Size

- **Drugs:** 8,263
- **TWOSIDES Interactions:** 42M+
- **Mapped Severities:** 18M+

Rule Engine Coverage

- **Specific Drugs:** 70+
- **Drug Classes:** 20+ patterns
- **eGFR Thresholds:** 3 levels (<15, <30, <45)

TROUBLESHOOTING

Frontend Not Loading Patients

- Check backend is running on port 8001
- Check `API_URL` in `DiabetesManager.jsx` (should be `http://localhost:8001`)
- Check browser console for errors

ML Predictions All Similar

- Check `optimal_threshold.json` exists
- Verify models are loaded (`models/*.pkl` files exist)
- Check class imbalance (use `--resampling smote`)

OCR Not Working

- Install Tesseract OCR
- Check `pytesseract` can find Tesseract
- Try different image preprocessing

Database Errors

- Check `data/interactions.db` exists
- Run `init_db()` to create tables
- Check SQLite file permissions

LEARNING RESOURCES

Key Concepts

1. **Class Imbalance:** 97% positive, 3% negative → Use class weights/SMOTE
2. **Optimal Threshold:** Don't use 0.5, use G-Mean/Youden's J
3. **NPV (Negative Predictive Value):** Critical for "safe" predictions
4. **Rules Primary, ML Supplementary:** Safety first!

Technologies Used

- **FastAPI:** Modern Python web framework
- **React:** Frontend framework
- **SQLAlchemy:** ORM for database
- **XGBoost/LightGBM:** Gradient boosting
- **Tesseract:** OCR engine
- **Tailwind CSS:** Utility-first CSS



SUMMARY

This is a **hybrid AI system** that:

1. **Prioritizes clinical rules** (safety first)
2. **Uses ML as supplementary** (learns from data)
3. **Handles class imbalance** (SMOTE, class weights, optimal threshold)
4. **Optimized for diabetic patients** (eGFR-based rules)
5. **Real-time risk assessment** (FastAPI + React)

The system is **production-ready** with:

-  Comprehensive rule engine
-  Optimized ML models (99% NPV)
-  OCR for medication labels
-  Patient management
-  Audit logging

Made with  for patient safety